Progettazione di un Digital Twin Cognitivo: Architettura Multi-Agente per la Gestione delle Smart City

Introduzione e Obiettivo

Nelle moderne infrastrutture urbane (Smart City), la massiccia proliferazione di sensori IoT genera un duplice ordine di criticità. Da un lato, si assiste a un drastico sovraccarico di allarme causato da un'elevata incidenza di falsi positivi (es. un forte rumore ambientale scambiato per uno sparo, o sensori ottici ingannati da riflessi).
Questo genera un enorme spreco di risorse pubbliche, costringendo Forze dell'Ordine e Vigili del Fuoco a interventi a vuoto.
Dall'altro lato, emerge la necessità cruciale di gestire le emergenze reali e confermate in maniera autonoma, tempestiva ed efficace, superando i colli di bottiglia e i fisiologici ritardi decisionali intrinseci nei tradizionali centri di dispacciamento gestiti da operatori umani.

Questo progetto nasce dalla necessità di risolvere tale inefficienza strutturale sviluppando un ecosistema distribuito capace di operare come un vero e proprio centro di comando cognitivo e autonomo.
Integrando paradigmi IoT con un'architettura di Agentic AI, il sistema crea un Digital Twin reattivo della città. Attraverso il ragionamento multimodale, l'infrastruttura persegue un doppio scopo: smentire i falsi positivi per preservare le risorse pubbliche e, simultaneamente, orchestrare in totale autonomia la risposta per gli incidenti confermati.

1.Topologia e Architettura di Sistema

L'architettura è strutturata su un modello multi-tier, progettato per garantire il disaccoppiamento tra il livello sensoriale periferico e il nucleo decisionale centralizzato.
Il grafico seguente illustra i flussi di comunicazione asincroni (tramite protocollo MQTT) e l'inclusione dei microservizi adibiti al triage dei dati.

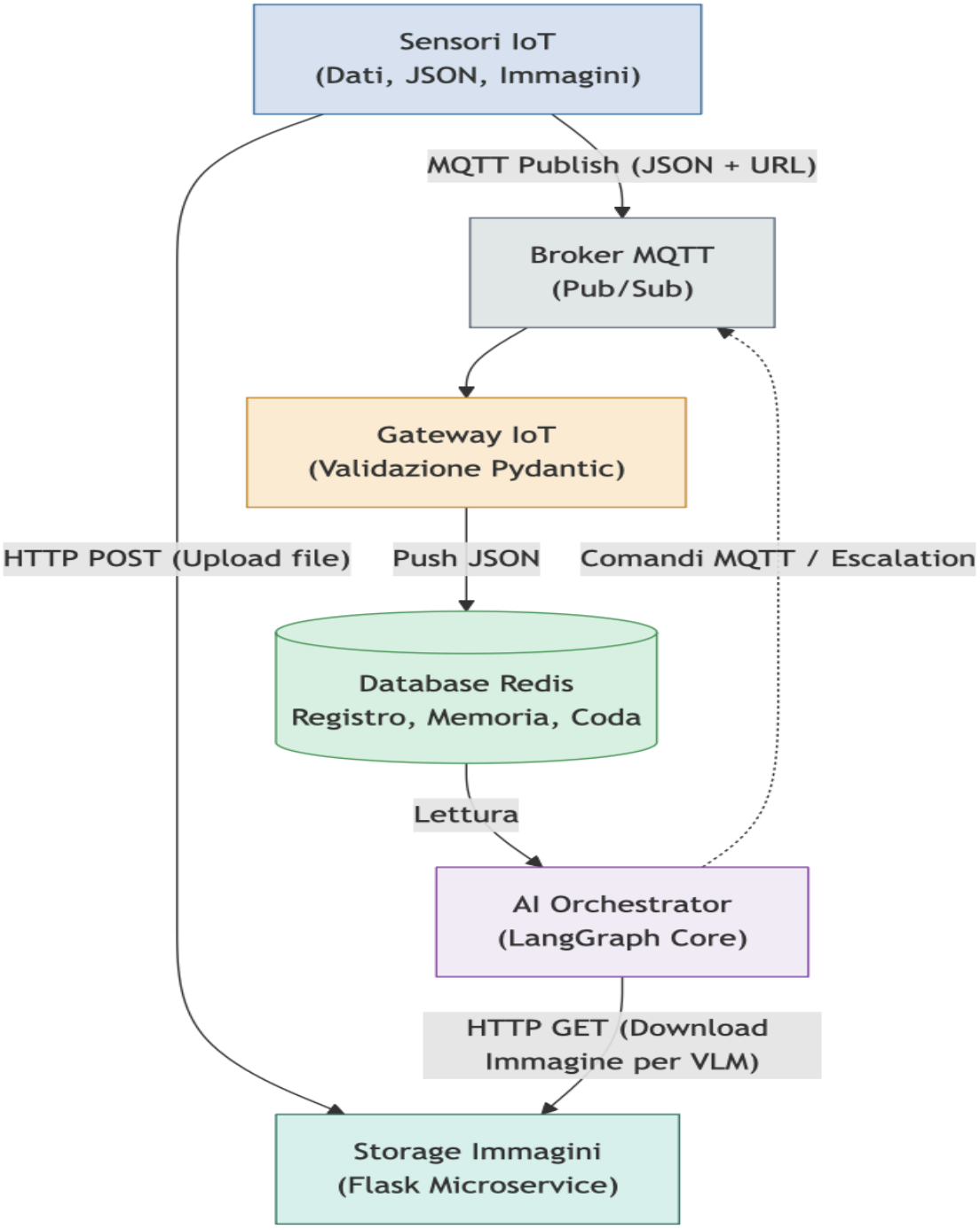


Figura 1 - Diagramma dell'architettura di sistema.

2. Moduli Core e Livelli di Elaborazione

2.1. Livello Ingress

Il livello di ingresso gestisce e dirama i flussi di dati provenienti dal broker MQTT:

Gateway_IoT.py applica lock transazionali su base geografica (Mutex Distribuito in Redis) per prevenire occorrenze di race condition qualora molteplici sensori della medesima area scatenino trigger simultanei.

Storage_Immagini.py è il microservizio dedicato all'ascolto dei feed visivi inviati dalle fotocamere. Si occupa di estrarre e salvare il dato binario, restituendo un URL referenziato. In questo modo l'AI Orchestrator non è appesantito dall'estrazione del payload raw.

2.2. Livello di Stato e Contesto

L'Intelligenza Artificiale necessita di un contesto spazio-temporale per ragionare efficacemente. L'ecosistema si appoggia a Redis in due modalità distinte:

Memoria Spazio-Temporale (memoria.py), i dati vengono inseriti in Sorted Sets indicizzati per timestamp. Una funzione ciclica elimina automaticamente i record obsoleti (es. antecedenti a 10 secondi), offrendo all'AI una fotografia netta ed in tempo reale della zona.

registro.py è stato implementato per garantire la localizzazione degli asset. I dispositivi inviano heartbeat periodici che ripristinano il Time-To-Live su Redis. Se un sensore smette di comunicare, la chiave decade e l'Orchestratore sa di non poter più contare su di esso.

3. Modelli dei Dati

Un aspetto cruciale per il corretto funzionamento del Gateway IoT è la standardizzazione e validazione dei messaggi periferici.
Il modulo **schema.py** descrive questo comportamento.
I payload JSON intercettati dal broker MQTT non vengono riversati in Redis così come sono, ma devono corrispondere a due schemi principali:

MessaggioIoT definisce lo standard per i rilevamenti attivi. Prevede la presenza obbligatoria di un *id_dispositivo*, una classificazione spaziale e un timestamp.
Inoltre, ammette attributi opzionali come l'url_immagine (valorizzato dal microservizio di storage) qualora il sensore sia di natura ottica.

SegnaleDiVita è uno schema alleggerito, progettato per ridurre l'overhead di rete, utilizzato esclusivamente per certificare la presenza in vita del nodo IoT sulla mappa logica in Redis.

In caso di anomalie di formattazione o parametri mancanti, Pydantic solleva un ValidationError gestito dal blocco try-except del Gateway, che provvederà a scartare il pacchetto impedendo il crash dell'AI Orchestrator.

4. Workflow Decisionale

Il nucleo cognitivo dell'infrastruttura è l'AI Orchestrator, sviluppato adottando il framework LangGraph. L'orchestrazione non si basa su rigide catene sequenziali, bensì su un grafo a stati condizionali capace di implementare loop di instradamento dinamico.

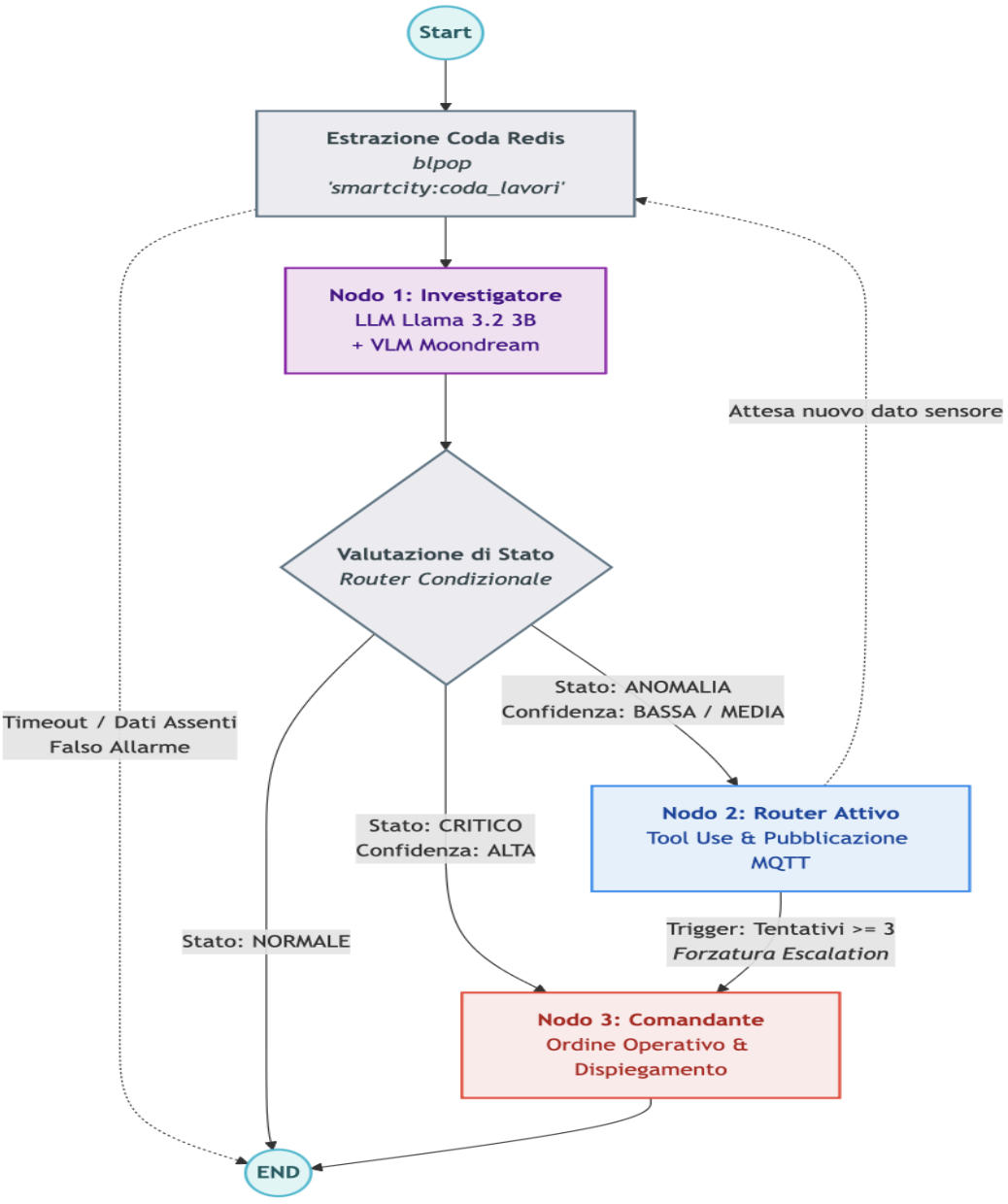


Figura 2 - Flowchart del processo decisionale

4.1 Multithreading

Per far fronte all''intensa mole di eventi generata da una Smart City, l'Orchestrator è stato implementato con un ThreadPoolExecutor parametrizzabile (attraverso la variabile WORKERS_COUNT).
Il ciclo principale estrae costantemente lavori dalla coda Redis quando arriva un evento, questo viene delegato a uno dei worker liberi
che eseguirà il grafo LangGraph in isolamento. Ciò permette al sistema di indagare, in parallelo e senza latenza incrociata, su distinte potenziali emergenze situate in macro-zone urbane differenti.

4.2. Nodo Investigatore e Controllo dell'LLM

Questo nodo rappresenta la percezione e il ragionamento di primo livello.
Il suo flusso di esecuzione si adatta dinamicamente al tipo di payload ricevuto.
Qualora l'evento disponga di un feed visivo (es. un URL immagine inviato da una telecamera), il nodo invoca preliminarmente un Vision Language Model (Moondream) per transcodificare l'immagine in un referto semantico testuale.
Se il referto è puramente testuale (es. un picco anomalo registrato da un microfono o da un sensore termico), il passaggio visivo viene bypassato.
In entrambi i casi, l'intero contesto viene poi sottoposto al ragionamento logico di Llama 3.2.
La stabilità di questo processo è garantita da rigidi vincoli di Prompt Engineering e validazione di output.
Il prompt istruisce esplicitamente il modello a priorizzare la smentita dei sensori (1. FALSO ALLARME) se la visione non corrobora i dati testuali.
Seguono poi i casi di (2. EMERGENZA CONFERMATA) e (3. VUOTO SENSORIALE) dove l'intelligenza è incerta.
Per evitare che l'LLM risponda con prosa discorsiva incontrollabile, il framework LangChain vincola la risposta ad un parser JSON. Il modello è obbligato a emettere un dizionario contenente chiavi esatte: stato_zona, livello_confidenza, e l'eventuale ID di un dispositivo_richiesto.

4.3. Nodi Router e Comandante

A seguito di quanto analizzato nella sezione precedente, se l'Investigatore produce confidenza "MEDIA/BASSA", interviene il Nodo Router. Questo modulo possiede capacità di Tool Use: interroga i dispositivi e pubblica comandi MQTT mirati.
Presenta un protocollo di Escalation che, superati i 3 tentativi di verifica infruttuosi, alza la gravità per non ignorare rischi latenti e indirizzando quindi il workflow verso il nodo comandante.

Il Nodo Comandante viene evocato esclusivamente quando sussistono due condizioni in contemporanea: la presenza di una criticità confermata (stato della zona identificato come "CRITICO" o "ANOMALIA") e un livello di confidenza "ALTO" (raggiunto fin dalla prima indagine o a seguito dell'escalation dal Router).

Se lo stato valutato dall'Investigatore è "NORMALE", l'iterazione di LangGraph termina immediatamente (raggiungendo il nodo END) per evitare sprechi operativi.

Il comportamento del Comandante è regolato da un rigoroso System Prompt, articolando la sua decisione come segue:
Valutazione del Contesto analizzando dinamicamente i dati spaziali, il report analitico prodotto dai modelli precedenti e la lista aggiornata dei dispositivi IoT disponibili in zona estratti dal database Redis.
Allocazione Ottimizzata delle Risorse valutando il dispiegamento tattico selezionando i servizi di emergenza strettamente necessari . Qualora l'intervento sia superfluo, è programmato per astenersi (output "NESSUNO").
Per garantire l'interoperabilità e la leggibilità con i sistemi di dispacciamento, l'output dell'LLM è vincolato, tramite Output Parsers, alla generazione di un payload JSON rigoroso. Questo referto include l'elenco delle unità mobilitate e gli ID dei dispositivi attivati.

5. Infrastruttura di Deploy e Tecnologie usate

L'implementazione pratica del sistema poggia su uno stack tecnologico orientato ai microservizi in un ambiente containerizzato.
Questo garantisce modularità, scalabilità e replicabilità in ambienti di produzione.

5.1. Containerizzazione

L'intero ecosistema è ingegnerizzato per essere distribuito tramite Docker. L'uso di Docker Compose permette di orchestrare simultaneamente l'avvio e la configurazione di tutti i microservizi (Gateway IoT, AI Orchestrator, Storage, Redis, Broker MQTT).
Questo approccio garantisce l'isolamento delle dipendenze, facilitando il deployment su architetture Edge.

5.2. Web Server

Il modulo **Storage_immagini.py** è implementato sfruttando Flask, un framework web leggero in Python. Flask agisce come server HTTP per ricevere i payload binari delle telecamere in modo rapido, salvando i file nel file system e restituendo immediatamente l'URL al broker, esponendo un'interfaccia RESTful snella e priva di latenze eccessive.

5.3. Inferenza AI Locale

Il motore di elaborazione è affidato a Ollama, che agisce come server di inferenza per i modelli Llama 3.2 e Moondream. Configurato per girare all'interno della rete Docker, Ollama permette l'esecuzione accelerata dell'intelligenza artificiale on-premise.
Questa scelta è fondamentale in quanto azzera i costi ricorrenti tipici delle API cloud.

6. Criticità Architetturali

Un'analisi approfondita del sistema ha evidenziato limitazioni operative relative alla gestione spaziale e alla categorizzazione hardware che dovranno essere risolte per elevare il grado di sicurezza e affidabilità dell'infrastruttura.

6.1. Risoluzione Geospaziale

Attualmente, la ricerca dei dispositivi ausiliari da interrogare viene effettuata basandosi esclusivamente sul parametro generico "zona" (es. nome_zona).
Questo approccio crea zone d'ombra: un'emergenza verificatasi esattamente al confine di due zone potrebbe non attivare le telecamere della zona adiacente, pur essendo fisicamente a pochi metri di distanza.

È imperativo migrare la localizzazione a un sistema geospaziale basato su coordinate GPS (latitudine/longitudine) inviate da ogni dispositivo.
Sfruttando le informazioni spaziali di seguito l'AI potrà richiedere l'attivazione dei dispositivi situati in un raggio esatto dall'epicentro.
Questo garantirà una flessibilità nettamente superiore e la certezza matematica di coinvolgere i sensori più prossimi all'evento.

6.2 Specificità del Payload JSON

Lo schema del messaggio JSON inviato dai moduli IoT manca di un descrittore specifico per la sottocategoria hardware o software del dispositivo.
Definire un nodo semplicemente come "microfono" o "telecamera" è insufficiente per un Agentic AI che deve ottimizzare l'ingaggio dei sensori in situazioni critiche.
Occorre estendere lo schema definito in Pydantic introducendo metadati di capacità (es. sub_tipo o capacita_specifiche).

Un microfono generico (in grado solo di rilevare decibel) non è utile quanto un microfono con algoritmi integrati capace di distinguere specificamente tra spari, urla umane o sirene.
L'AI deve poter richiedere l'ascolto esplicitamente al secondo tipo per validare una rapina.

Una telecamera standard produce fotogrammi ambientali, mentre una telecamera dotata di Computer Vision specializzata può rilevare attivamente armi da fuoco, principio d'incendio o mezzi di soccorso.

Introducendo questi attributi nel Payload, il Nodo Router potrà interrogare chirurgicamente solo i dispositivi dotati dei requisiti esatti per confermare un dubbio specifico, massimizzando il rendimento dell'interrogazione MQTT attiva.